

Informe de Arquitectura

Plataforma Web de Cursos Masivos en Línea (MOOC) — Entrega 1

Fecha de entrega: 15 de septiembre de 2026

Repositorio: github.com/danielasolarte/Proyecto1-Desarrollo

Integrantes
Daniela Solarte
Samara Martínez
Andrés Jurado (ajustar nombre completo)
Camila Fajardo

1. Resumen ejecutivo

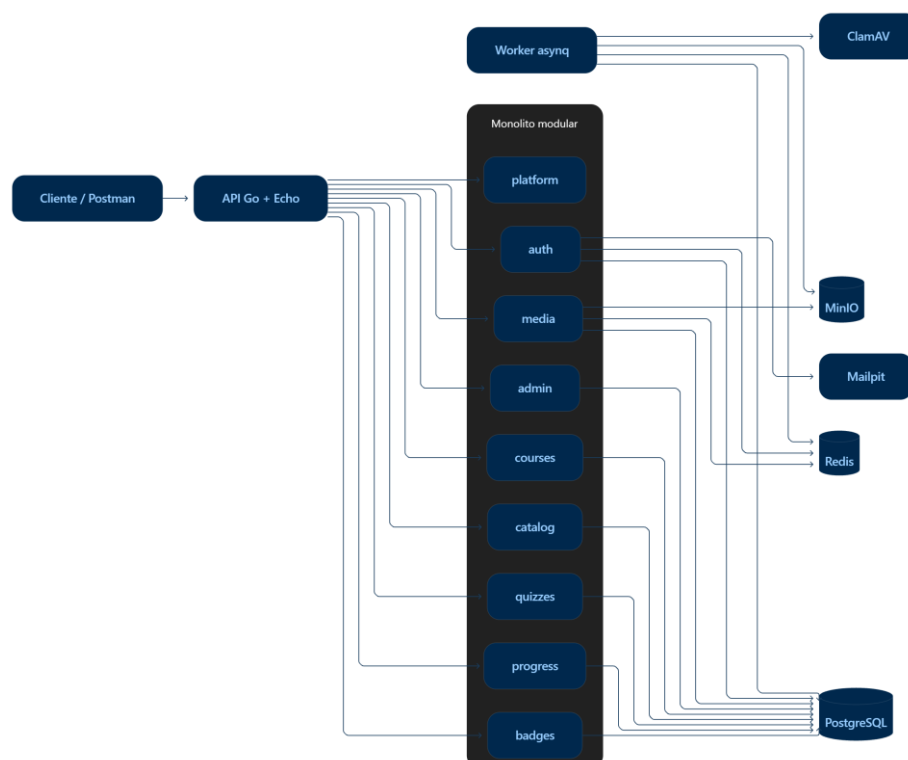
En esta entrega se construyó el backend de una plataforma MOOC mediante una arquitectura de monolito modular en Go, cubriendo los frentes de identidad y administración, autoría y catálogo de cursos, multimedia, evaluación mediante quizzes, seguimiento de progreso e insignias. La mayor parte de las funcionalidades obligatorias se encuentra implementada y disponible mediante una API REST bajo /api/v1, con persistencia en PostgreSQL, Redis para sesiones y trabajos asíncronos, y MinIO para almacenamiento de objetos. Los módulos de cursos, identidad, quizzes, progreso e insignias cuentan con pruebas funcionales o automáticas, mientras que multimedia presenta una cobertura de pruebas más limitada. Para esta entrega se dejó fuera de alcance una interfaz gráfica completa y la observabilidad avanzada mediante OpenTelemetry, métricas Prometheus y tracing distribuido. También se reconocen como pendientes mecanismos adicionales como rate limiting y protecciones específicas contra CSRF.

2. Arquitectura general del sistema

2.1 Estilo arquitectónico

- Monolito modular en Go con workers asíncronos independientes (asynq), tal como exige la sección 4 del enunciado del proyecto.
- Cada módulo de dominio sigue el mismo patrón de capas, para desacoplar el dominio del framework HTTP y del proveedor de nube:
 - domain/ — entidades, errores e interfaces del módulo (sin dependencias externas).
 - usecase/ — reglas de negocio.
 - postgres/ — implementación de los repositorios contra PostgreSQL (pgx/v5).
 - http/ — handlers y rutas Echo.
- La API y los workers no mantienen estado local; el estado vive en PostgreSQL, Redis y MinIO.

2.2 Diagrama de componentes



La aplicación se organiza como un monolito modular. Los módulos comparten el mismo proceso de API, pero mantienen separación por dominio y capas internas. PostgreSQL funciona como fuente de verdad, Redis soporta sesiones y la cola asíncrona, y MinIO almacena objetos multimedia. Los trabajos pesados son procesados por workers independientes mediante asynq, permitiendo escalamiento horizontal sin duplicar la lógica del dominio.

2.3 Stack tecnológico

Componente	Tecnología	Rol en el sistema
API HTTP	Go + Echo	Enrutamiento REST bajo /api/v1, autenticación, validación
Base de datos transaccional	PostgreSQL (driver pgx/v5)	Fuente de verdad de todos los módulos
Cache, sesiones y cola	Redis + asynq	Sesiones revocables, cache, cola de trabajos asíncronos
Almacenamiento de objetos	MinIO (compatible S3)	Originales y derivados multimedia, URLs prefiradas
Correo de desarrollo	Mailpit	Verificación de correo y recuperación de contraseña en local
Antimalware	ClamAV	Escaneo de archivos subidos por el worker de multimedia
Migraciones	golang-migrate	Versionado del esquema SQL (4 migraciones aplicadas)
Contrato de API	OpenAPI 3.1 (api/openapi.yaml)	54 endpoints documentados

2.4 Decisiones de diseño clave

Se eligió un monolito modular debido al tamaño del equipo, el plazo corto de desarrollo y la necesidad de mantener una integración sencilla entre dominios altamente relacionados. Separar cada módulo como microservicio habría agregado complejidad en despliegue, comunicación distribuida, observabilidad, consistencia de datos y coordinación entre servicios. El monolito modular permite conservar límites claros entre dominios mediante interfaces y capas, manteniendo al mismo tiempo un único despliegue y una sola base de datos transaccional. Esto permite que una futura evolución hacia servicios independientes sea posible sin asumir prematuramente el costo operativo de una arquitectura distribuida.

Se eligió asynq porque Redis ya era parte de la arquitectura para sesiones y cache, evitando incorporar una nueva pieza de infraestructura únicamente para el procesamiento asíncrono. Asynq proporciona colas, reintentos, concurrencia y procesamiento distribuido con una integración sencilla en Go. Para el alcance de esta entrega, resultaba más liviano que RabbitMQ y más fácil de ejecutar localmente que alternativas administradas como SQS, manteniendo la posibilidad de escalar varios workers sobre la misma cola.

MinIO se eligió porque proporciona una API compatible con S3 y puede ejecutarse completamente en local mediante Docker Compose. Esto permite desarrollar y probar carga de archivos, URLs prefiradas y almacenamiento de derivados multimedia sin depender de credenciales o servicios de

nube externos. La compatibilidad con S3 también reduce el acoplamiento, ya que una migración futura hacia un proveedor cloud requeriría cambios mínimos en la capa de almacenamiento.

Por restricciones de tiempo se priorizó completar los flujos funcionales principales y sus validaciones de seguridad antes que incorporar componentes operativos adicionales. No se implementó rate limiting, protección CSRF específica ni tracing distribuido con OpenTelemetry. La API sí produce logs estructurados en formato JSON mediante Echo, mientras que los workers utilizan logs operativos de aplicación y Asynq. La instrumentación avanzada mediante métricas Prometheus, dashboards y trazas distribuidas queda declarada como mejora para una siguiente iteración.

3. Modelo de datos

[Completar equipo] Insertar diagrama entidad-relación o lista de tablas con sus relaciones. Explicar por qué se usan identificadores estables (stable_id) en módulos, unidades y recursos para preservar el progreso del estudiante cuando se publica una nueva versión de un curso.

3.1 Tablas por migración

- 000001_create_courses_catalog_tables — courses, course_versions, modules, units, resources, enrollments.
- 000002_create_auth_admin_tables — users, sessions, audit_log.
- 000003_create_quizzes_progress_badges_tables — quizzes, questions, question_options, attempts, attempt_answers, resource_progress, progress_events, course_progress, badges, badge_issuances.
- 000004_create_media_tables — activos y trabajos de procesamiento multimedia.

4. Módulos del sistema

4.1 Persona 1 — Autoría de cursos y Catálogo (Daniela Solarte)

Alcance implementado

- Jerarquía Curso → Módulo → Unidad → Recurso con metadatos, posiciones ordenadas e identificadores estables.
- Estados draft / published / superseded, con versiones publicadas inmutables.
- Previsualización de curso con lista exhaustiva de problemas de publicación.
- Editor Markdown para recursos rich_text, con autosave y recuperación del borrador.
- Catálogo público de cursos publicados con búsqueda, filtros y paginación por cursor.
- Inscripción, retiro y reinscripción conservando historial.

Nota de verificación: colección Postman propia

(postman/MOOC_Persona1.postman_collection.json), 37 requests en 4 carpetas — 37/37 passing en la última corrida.

Decisiones de diseño

Versionar con course_versions en vez de mutar el curso publicado directamente responde a que un curso publicado puede tener estudiantes inscritos en ese momento. Si se editara la versión publicada en el sitio, un estudiante a mitad de una unidad podría ver contenido que cambia, se reordena o desaparece sin aviso, en plena sesión. Por eso Course guarda dos punteros separados, published_version_id (lo que ven los estudiantes) y latest_draft_version_id (lo que edita el profesor): son entidades distintas y nunca se pisan entre sí. Este diseño además habilita el preview con validación de problemas antes de publicar, sin tocar lo que ya está en producción, y deja historial real: al publicar un borrador nuevo, la versión anterior pasa a superseded en vez de borrarse, preservando trazabilidad de qué vio cada estudiante y

cuándo. El costo es más complejidad de modelo frente a un curso mutable, pero es lo que exige la sección 7 del enunciado sobre versiones publicadas inmutables.

El campo `change_type` (minor/major) se guarda como metadato que el profesor declara al abrir un borrador de actualización, pero en esta entrega no hay lógica de negocio que se comporte distinto según su valor: no dispara notificaciones a estudiantes inscritos, no cambia validaciones ni el flujo de publicación. Es un punto de extensión dejado listo (el enum y la columna existen, con `omitempty` en el JSON) para una funcionalidad futura, típicamente notificar a los inscritos solo cuando el cambio es mayor. Fue una decisión consciente de alcance para esta entrega, no un olvido: el dato queda capturado para cuando se necesite, pero no se invirtió tiempo en la lógica diferenciada porque no estaba entre los 10 frentes obligatorios de la sección 5.1.

4.2 Persona 2 — Identidad y Administración (Samara Martínez)

Alcance implementado

El módulo de Identidad y Administración implementa el registro público de estudiantes, verificación de correo mediante Mailpit, login con contraseñas hasheadas con `bcrypt`, recuperación de contraseña por token y sesiones opacas revocables. Las sesiones se persisten en PostgreSQL y se cachean en Redis para validación rápida, manteniendo la posibilidad de revocarlas de forma inmediata.

También se implementó el bootstrap del primer administrador, la creación de profesores y administradores desde endpoints administrativos, la consulta y actualización de usuarios, el manejo de roles y estados, la revocación individual y masiva de sesiones, y el middleware real de autenticación y autorización por roles. El módulo incluye auditoría persistente en la tabla `audit_log`, protegida como `append-only` mediante triggers de base de datos que impiden `UPDATE` y `DELETE`.

Verificación

Nota de verificación: La verificación del módulo se realizó mediante pruebas automáticas con `go test` y una colección de Postman para Persona 2. Las pruebas automáticas cubren registro de estudiante, verificación de correo, login, protección del último administrador activo y middleware HTTP de autenticación y roles.

En Postman se validaron los siguientes flujos: `healthcheck` de la API, creación del primer administrador, login de administrador, acceso a rutas administrativas, creación de profesor, login de profesor, registro de estudiante, verificación de correo, login de estudiante, consulta del usuario autenticado, rechazo de estudiante en rutas admin, rechazo de rutas sin token, consulta administrativa de usuarios, listado y revocación de sesiones, suspensión/reactivación de estudiante, recuperación de contraseña, login con nueva contraseña y consulta de auditoría.

Decisiones de diseño

Se eligieron sesiones opacas revocables en lugar de JWT porque el proyecto exige revocación inmediata de sesiones. Con JWT, una vez emitido el token, este suele ser válido hasta su expiración salvo que se agregue una lista negra o un mecanismo adicional de invalidación. En cambio, con sesiones opacas el cliente solo posee un token aleatorio, mientras que el estado real de la sesión vive en PostgreSQL y Redis. Esto permite cerrar sesión, revocar sesiones desde administración o suspender usuarios de forma inmediata.

Redis se usa como cache de sesión para acelerar la validación de tokens en cada request, mientras PostgreSQL permanece como fuente de verdad. Si Redis pierde la sesión cacheada, el sistema puede consultar PostgreSQL y reconstruir la cache si la sesión sigue vigente y no ha sido revocada.

La auditoría se diseñó como `append-only` para que los eventos críticos no puedan ser modificados o eliminados después de escritos. Esto no depende únicamente de la lógica de aplicación: la tabla

`audit_log` incluye triggers en PostgreSQL que bloquean operaciones UPDATE y DELETE. De esta forma, acciones como registro, login, verificación de correo, recuperación de contraseña, creación de profesores/admins, cambios de estado y revocación de sesiones quedan registradas como evidencia persistente.

4.3 Persona 3 — Multimedia (Andrés Jurado)

Alcance implementado

- Generación de URLs prefiradas contra MinIO y carga multipart reanudable, con reanudación basada en lo que el storage confirma que ya llegó y no en lo que reporte el cliente.
- Verificación de checksum declarado contra checksum real y de MIME real por firma binaria, no por extensión declarada ni por el Content-Type que envía el cliente al iniciar la carga.
- Integración con ClamAV para escaneo antimalware del archivo completo ya descargado y reensamblado por el worker.
- Worker asynq más ffmpeg para transcodificación a HLS, con conservación del original y sin upscaling de resolución.
- Endpoint de reproducción con URL firmada, validando inscripción activa del estudiante.
- Tolerancia a fallos mediante reintentos con backoff y paso a dead letter tras tres intentos fallidos, evitando que una entrega duplicada genere salidas repetidas gracias a la idempotencia por asset y tipo de job.

Decisiones de diseño

Se genera una sola resolución de HLS por video, sin adaptive bitrate multi-resolución: mismo códec H.264/AAC, misma resolución del original, segmentos de 6 segundos. La decisión fue de alcance para el MVP, ya que generar múltiples resoluciones multiplica el tiempo de transcodificación y el almacenamiento por cada rendition adicional, sin estar entre los diez frentes obligatorios de la sección 5.1. El código evita además cualquier upscaling, nunca forzando una resolución de salida mayor a la de entrada, cumpliendo la condición de la sección 10.2 sobre HLS sin upscaling con un único perfil.

El MIME se verifica por firma binaria después de la descarga completa por dos razones. Primero, ni el Content-Type declarado por el cliente ni el ETag de MinIO/S3 son confiables: el cliente puede declarar cualquier valor al iniciar la carga, y el ETag de una carga multipart en S3/MinIO no es un hash simple del archivo final sino un hash de hashes de las partes, por lo que tampoco sirve para verificar integridad o tipo de contenido real. Segundo, la detección de MIME por firma necesita los primeros bytes del archivo ya ensamblado; en una carga multipart ninguna parte individual contiene el archivo completo, así que la única forma correcta de inspeccionar el contenido real es después de que el storage reconstruye el objeto completo, que es justo el momento en que corre el job `scan_antivirus` en el worker. Se usó Redis con asynq para la cola de trabajos en vez de una cola dedicada, reutilizando la misma pieza de infraestructura que ya soporta sesiones y cache en el resto del sistema, y evitando sumar un componente operativo adicional para un único frente del MVP.

4.4 Persona 4 — Evaluación, Progreso e Insignias (Camila Fajardo)

Alcance implementado

- Quizzes de selección múltiple con snapshot generado al iniciar el intento, sin exponer al estudiante información sobre las respuestas correctas.
- Guardado parcial de respuestas y recuperación de intentos en progreso.
- Calificación y porcentaje calculados exclusivamente en el servidor.
- Submit final protegido mediante Idempotency-Key, de forma que repetir la misma operación no genere una segunda calificación ni efectos duplicados.

- Progreso por recurso construido mediante eventos y heartbeats validados en servidor, sin aceptar porcentajes calculados directamente por el cliente.
- Validación de tiempo activo, posición multimedia y consistencia temporal de los heartbeats.
- Cálculo de completed cuando todos los recursos obligatorios están completos y de approved cuando, adicionalmente, los quizzes requeridos han sido aprobados.
- Uso de identificadores estables para preservar progreso entre versiones del curso.
- Insignias con emisión única por estudiante y curso, código público de verificación, ausencia de información sensible en la respuesta pública y revocación administrativa.

Verificación: colección Postman propia (postman/MOOC_Persona4_Postman.json) que cubre el flujo de quizzes, cálculo de progreso, emisión y verificación de insignias, control de acceso por rol y rechazo de heartbeats con posiciones inválidas.

Decisiones de diseño

Se utilizó Idempotency-Key porque el submit es una operación crítica que puede ser reenviada por problemas de red, timeouts o reintentos del cliente. La clave permite reconocer que dos solicitudes representan la misma intención del usuario y devolver el resultado previamente calculado sin volver a calificar ni producir efectos secundarios duplicados. Esto resulta más robusto que depender únicamente del estado del intento o del timestamp de la solicitud, y además permite garantizar el comportamiento mediante una restricción única en PostgreSQL.

Para recursos normales se definió un mínimo de 30 segundos de actividad válida antes de permitir marcarlos como completados. El objetivo es evitar que un cliente pueda abrir un recurso y marcarlo inmediatamente como terminado sin haber interactuado con él. El tiempo activo se construye a partir de heartbeats aceptados por el servidor, con límites sobre el intervalo entre eventos y validaciones de posición. En recursos multimedia, además del tiempo mínimo, se exige alcanzar al menos el 90 % de la duración conocida del contenido.

5. Seguridad y control de acceso transversal

5.1 Verificado en el código actual

- Cada endpoint sensible exige rol, propiedad o inscripción antes de responder (con la excepción anotada en 4.3).
- La clave correcta de un quiz nunca aparece en ninguna respuesta de la API.
- El porcentaje de progreso enviado directamente por el cliente se rechaza; solo cuenta lo calculado en servidor.
- Una doble entrega del mismo trabajo (transcodificación o submit de quiz) no produce efectos duplicados, respaldado por índices únicos en PostgreSQL.

5.2 No encontrado en el código actual

No implementado o implementado parcialmente

- Rate limiting: no implementado en esta entrega.
- Protección CSRF específica: no implementada. La API utiliza autenticación basada en tokens/sesiones enviados explícitamente por el cliente y no una interfaz web tradicional basada exclusivamente en cookies.
- Protección XSS: no existe una capa específica de sanitización de frontend, dado que el alcance actual corresponde principalmente al backend API.

- Logs estructurados: implementados en la API mediante el middleware de Echo, incluyendo timestamp, método, URI, estado y latencia.
- Observabilidad avanzada: no se implementaron métricas Prometheus, endpoint /metrics, OpenTelemetry ni trazas distribuidas.

Para esta entrega se priorizó completar los flujos funcionales, las reglas de negocio y los controles de autorización. Rate limiting y observabilidad avanzada quedan declarados como mejoras para una siguiente iteración.

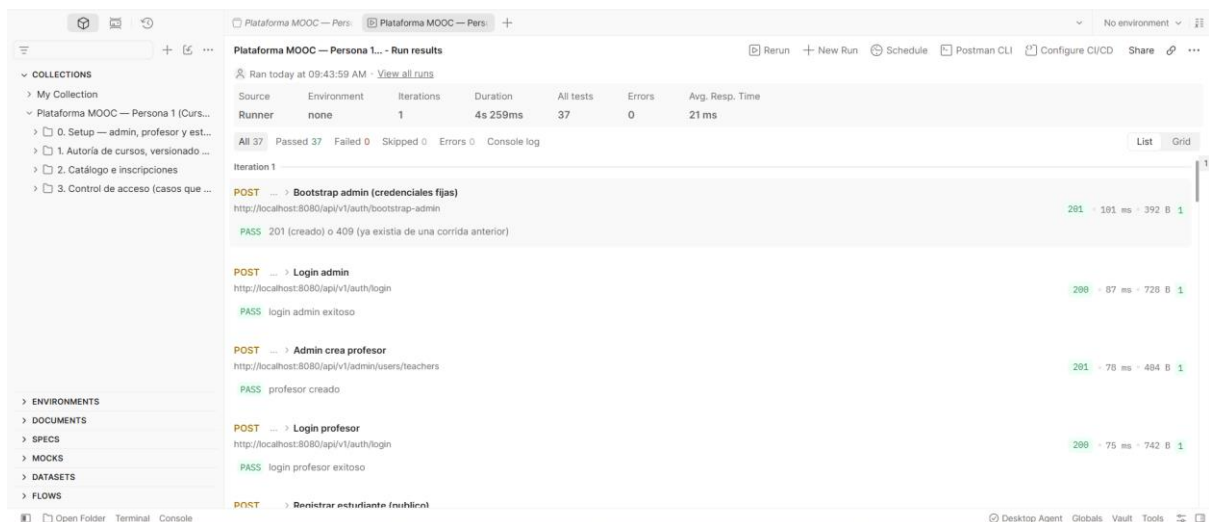
6. Despliegue

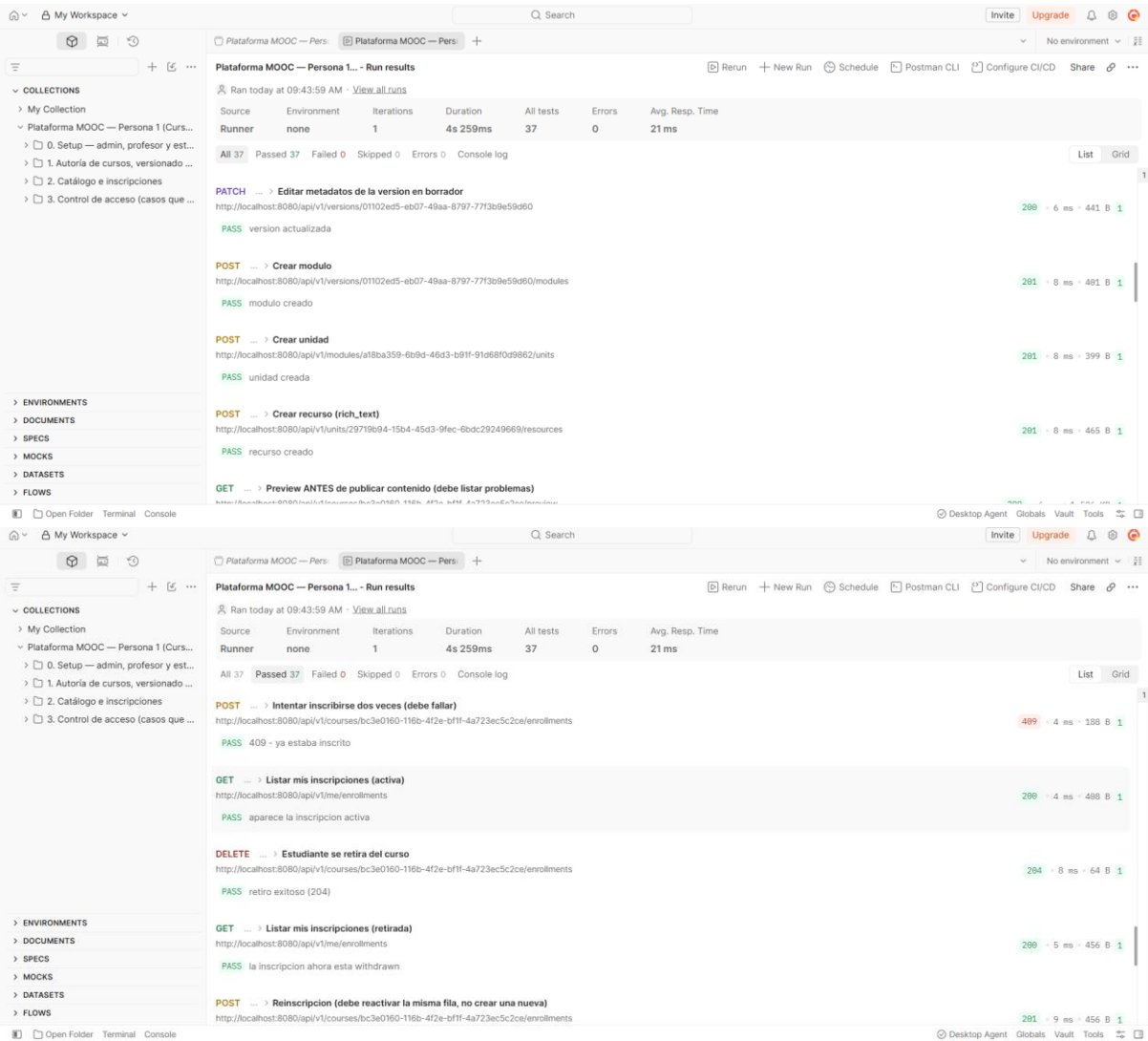
- docker-compose.yml orquesta: postgres, redis, minio, minio-init (crea el bucket y sus políticas), mailpit, clamav, api y worker.
- Dockerfiles existentes: deploy/Dockerfile.api, deploy/Dockerfile.worker.
- Variables de configuración (DATABASE_URL, REDIS_ADDR, S3_*, SMTP_*, CLAMAV_*) se inyectan por entorno, separadas del código.

7. Pruebas y validación

- go test ./... — cobertura automática actual: solo internal/auth (registro, verificación, login, protección del último admin, middleware).
- postman/MOOC_Persona1.postman_collection.json — 37 requests, cursos/autoría/catálogo.
- postman/MOOC_Persona4_Postman.json — quizzes, progreso e insignias.

Persona 1:





Persona 2:

1. Setup y Admin

GET 01 - Healthcheck	http://localhost:8080/health	200 OK	8 ms	124 B
PASS	API responde healthcheck			
PASS	status ok			
POST 02 - Crear primer admin	http://localhost:8080/api/v1/auth/bootstrap-admin	409 Conflict	113 ms	190 B
PASS	bootstrap crea admin o ya existia			
POST 03 - Login admin	http://localhost:8080/api/v1/auth/login	200 OK	589 ms	754 B
PASS	login admin 200			
PASS	usuario admin activo			
GET 04 - Listar usuarios admin	http://localhost:8080/api/v1/admin/users	200 OK	56 ms	12.055 KB
PASS	admin puede listar usuarios			
PASS	respuesta es arreglo			

2. Administración de usuarios

POST 05 - Crear profesor

http://localhost:8080/api/v1/admin/users/teachers

201 Created 918 ms 413 B

PASS profesor creado

PASS rol teacher activo

POST 06 - Login profesor

http://localhost:8080/api/v1/auth/login

200 OK 644 ms 769 B

PASS login profesor 200

PASS usuario teacher activo

POST 07 - Crear segundo admin

http://localhost:8080/api/v1/admin/users/admins

201 Created 534 ms 412 B

PASS segundo admin creado

PASS rol admin activo

3. Estudiante y control de acceso

POST 08 - Registrar estudiante

http://localhost:8080/api/v1/auth/register

201 Created 953 ms 506 B

PASS estudiante registrado

PASS estudiante pendiente de verificacion

POST 09 - Verificar correo estudiante

http://localhost:8080/api/v1/auth/verify-email

200 OK 47 ms 463 B

PASS correo verificado

PASS estudiante activo

POST 10 - Login estudiante

http://localhost:8080/api/v1/auth/login

200 OK 374 ms 824 B

PASS login estudiante 200

PASS usuario student activo

GET 11 - Me estudiante

http://localhost:8080/api/v1/me

200 OK 23 ms 462 B

PASS me estudiante 200

PASS retorna el estudiante

GET 12 - Estudiante no puede entrar a admin

http://localhost:8080/api/v1/admin/users

403 Forbidden 16 ms 196 B

PASS estudiante recibe forbidden

GET 13 - Ruta sin token

http://localhost:8080/api/v1/me

401 Unauthorized 10 ms 179 B

4. Sesiones, estados y recuperacion

GET 14 - Consultar estudiante en admin http://localhost:8080/api/v1/admin/users/ae14fe54-99b1-46c2-be66-54e2ed1fab1a	200 OK 20 ms 462 B
PASS admin consulta estudiante	
PASS id coincide	
GET 15 - Listar sesiones estudiante http://localhost:8080/api/v1/admin/users/ae14fe54-99b1-46c2-be66-54e2ed1fab1a/sessions	200 OK 22 ms 350 B
PASS lista sesiones	
PASS respuesta de sesiones es arreglo	
PASS hay al menos una sesion	
DELETE 16 - Revocar sesion individual http://localhost:8080/api/v1/admin/sessions/6b2bf4f7-3be1-48d8-8103-0b8cb23f3fab	204 No Content 43 ms 64 B
PASS sesion revocada	
GET 17 - Confirmar token revocado http://localhost:8080/api/v1/me	401 Unauthorized 9 ms 179 B
PASS token revocado no funciona	
POST 18 - Volver a hacer login estudiante http://localhost:8080/api/v1/auth/login	200 OK 303 ms 824 B
PASS login estudiante nuevo 200	
PATCH 19 - Suspende estudiante http://localhost:8080/api/v1/admin/users/ae14fe54-99b1-46c2-be66-54e2ed1fab1a	200 OK 49 ms 465 B
PASS estudiante suspendido	
PASS status suspended	
POST 20 - Login estudiante suspendido http://localhost:8080/api/v1/auth/login	403 Forbidden 342 ms 175 B
PASS suspendido no puede hacer login	
PATCH 21 - Reactivar estudiante http://localhost:8080/api/v1/admin/users/ae14fe54-99b1-46c2-be66-54e2ed1fab1a	200 OK 60 ms 462 B
PASS estudiante reactivado	
PASS status active	

5. Password reset y auditoria

POST 22 - Solicitar recuperacion de contrasena http://localhost:8080/api/v1/auth/password/forgot	202 Accepted 103 ms 81 B
PASS solicitud aceptada	
GET 23 - Mailpit listar correos http://localhost:8025/api/v1/messages	200 OK 51 ms 12.798 KB
PASS mailpit responde	
PASS hay correos en Mailpit	
GET 24 - Mailpit obtener token reset http://localhost:8025/api/v1/message/4qv4qhKVQqGePN97mUTVfv	200 OK 8 ms 948 B
PASS mensaje de reset obtenido	
PASS token de reset encontrado	
POST 25 - Cambiar contrasena http://localhost:8080/api/v1/auth/password/reset	204 No Content 385 ms 64 B
PASS contrasena cambiada	
POST 26 - Login con nueva contrasena http://localhost:8080/api/v1/auth/login	200 OK 366 ms 823 B
PASS login con nueva contrasena	
GET 27 - Auditoria http://localhost:8080/api/v1/admin/audit-log	200 OK 47 ms 45.415 KB
Running request...	

 Mailpit

Search mailbox

50

1-28 of 28

no-reply@mooc.local

16 minutes ago

To: ana.1789531655426@example.com

Recupera tu clave

Token de recuperacion: OHqBuEC4KSKe9xl4g82KdKAbfD6kQpcr6FYi2jvzT0U

no-reply@mooc.local

16 minutes ago

To: ana.1789531655426@example.com

Verifica tu correo

Token de verificacion: VYsUVEJM_O5cNjLaU_JRfXtDO40hGwmUcl9J-1vWwFo

no-reply@mooc.local

18 minutes ago

To: ana.1789531496169@example.com

Recupera tu clave

Token de recuperacion: 1Ubl1tJl6cc0nqJ8HE9aR84Uk6u7ny5xA3KvFCMILLA

no-reply@mooc.local

19 minutes ago

To: ana.1789531496169@example.com

Verifica tu correo

Token de verificacion: 1S5oEP_HhC4QDOOWMpOxLFUBoPz1JdwnH3fEu98Q1_A

no-reply@mooc.local

21 minutes ago

To: ana.1789531344947@example.com

Recupera tu clave

Token de recuperacion: bVxXi-aM-IGiNv_p1YQMV84legUfLUov8E-YSB-T1vl

no-reply@mooc.local

21 minutes ago

To: ana.1789531344947@example.com

Verifica tu correo

Token de verificacion: aqInJyV-r8W2VqMqCG6HAaLRO_gklgg95FA3ZTRhjtA

no-reply@mooc.local

28 minutes ago

To: ana.1789530926121@example.com

Recupera tu clave

Token de recuperacion: WE9T2eiKYbLWfkDHAEKmHrv10a5f-7N30ssPtzHzsBE

no-reply@mooc.local

28 minutes ago

Persona 3

Persona 4:

0. Setup – admin, profesor, estudiante

Iteration 1		1	
POST	0. Setup — admin, profesor y estudiante / Bootstrap admin (201 o 409) http://localhost:8080/api/v1/auth/bootstrap-admin	201	• 2317 ms • 410 B • 1
PASS	201 creado o 409 ya existía		
POST	0. Setup — admin, profesor y estudiante / Login admin http://localhost:8080/api/v1/auth/login	200	• 473 ms • 762 B • 1
PASS	login admin exitoso		
POST	0. Setup — admin, profesor y estudiante / Admin crea profesor http://localhost:8080/api/v1/admin/users/teachers	201	• 664 ms • 413 B • 1
PASS	profesor creado		
POST	0. Setup — admin, profesor y estudiante / Login profesor http://localhost:8080/api/v1/auth/login	200	• 174 ms • 769 B • 1
PASS	login profesor		
POST	0. Setup — admin, profesor y estudiante / Registrar estudiante http://localhost:8080/api/v1/auth/register	201	• 896 ms • 518 B • 1
PASS	estudiante registrado		
POST	0. Setup — admin, profesor y estudiante / Verificar correo estudiante http://localhost:8080/api/v1/auth/verify-email	200	• 77 ms • 474 B • 1
PASS	correo verificado		
POST	0. Setup — admin, profesor y estudiante / Login estudiante http://localhost:8080/api/v1/auth/login	200	• 554 ms • 835 B • 1
PASS	login estudiante		

Iteration 1			
POST	1. Autoría — curso, quiz e insignia / Crear curso		
http://localhost:8080/api/v1/courses		201	• 37 ms • 780 B • 1
PASS	curso y draft creados		
POST	1. Autoría — curso, quiz e insignia / Crear módulo		
http://localhost:8080/api/v1/versions/eabc25ba-5267-41ee-b6bb-7ea1759eddb2/modules		201	• 30 ms • 497 B • 1
PASS	módulo creado		
POST	1. Autoría — curso, quiz e insignia / Crear unidad		
http://localhost:8080/api/v1/modules/f0ccba60-a1dd-404a-bfc9-f271f8998c7/units		201	• 29 ms • 393 B • 1
PASS	unidad creada		
POST	1. Autoría — curso, quiz e insignia / Crear recurso quiz obligatorio		
http://localhost:8080/api/v1/units/4e6d09c8-92aa-4cbc-b97d-05a8d6085053/resources		201	• 36 ms • 454 B • 1
PASS	recurso quiz creado		
POST	1. Autoría — curso, quiz e insignia / Crear quiz		
http://localhost:8080/api/v1/resources/8ed01545-021a-4131-ad89-4a98a8a760da/quiz		201	• 45 ms • 393 B • 1
PASS	quiz creado		
POST	1. Autoría — curso, quiz e insignia / Crear pregunta		
http://localhost:8080/api/v1/quizzes/595b6f0e-a737-4600-983f-5fe16a816733/questions		201	• 30 ms • 418 B • 1
PASS	pregunta creada		
POST	1. Autoría — curso, quiz e insignia / Crear opción correcta		
http://localhost:8080/api/v1/questions/00303109-dc3a-4e33-8de1-cb5ff0755dea/options		201	• 39 ms • 347 B • 1
PASS	opción correcta creada		

POST	1. Autoría — curso, quiz e insignia / Crear opción incorrecta				
http://localhost:8080/api/v1/questions/00303109-dc3a-4e33-8de1-cb5ff0755dea/options					
		201	• 23 ms • 348 B •	1	
PASS opción incorrecta creada					
GET	1. Autoría — curso, quiz e insignia / Consultar quiz de autoría				
http://localhost:8080/api/v1/quizzes/595b6f0e-a737-4600-983f-5fe16a816733/authoring					
		200	• 87 ms • 1.18 KB •	1	
PASS autoría incluye opciones y clave					
POST	1. Autoría — curso, quiz e insignia / Crear badge del curso				
http://localhost:8080/api/v1/courses/885fdeb-76eb-43dd-9e74-23117f658640/badge					
		201	• 19 ms • 441 B •	1	
PASS badge creado					
POST	1. Autoría — curso, quiz e insignia / Publicar curso				
http://localhost:8080/api/v1/courses/885fdeb-76eb-43dd-9e74-23117f658640/publish					
		200	• 32 ms • 514 B •	1	
PASS curso publicado					

2. Estudiante – inscripción, intento, submit, progreso

Iteration 1

POST

2. Estudiante — inscripción, intento, submit y progreso / **Inscribirse al curso**

http://localhost:8080/api/v1/courses/885fdeb-76eb-43dd-9e74-23117f658640/enrollments

PASS inscripción activa

201

• 35 ms • 426 B •

1

POST

2. Estudiante — inscripción, intento, submit y progreso / **Iniciar intento**

http://localhost:8080/api/v1/quizzes/595b6f0e-a737-4600-983f-5fe16a816733/attempts

PASS intento iniciado

PASS snapshot NO expone is_correct

201

• 55 ms • 708 B •

2

PUT

2. Estudiante — inscripción, intento, submit y progreso / **Guardar respuesta parcial correcta**

http://localhost:8080/api/v1/attempts/ed424030-91cd-4762-8651-a1c3764b0654/answers/00303109-dc3a-4e33-8de1-cb5ff0755dea

PASS respuesta parcial guardada

200

• 1322 ms • 455 B •

1

GET

2. Estudiante — inscripción, intento, submit y progreso / **Recuperar intento (resume)**

http://localhost:8080/api/v1/attempts/ed424030-91cd-4762-8651-a1c3764b0654

PASS intento recuperable con respuestas

200

• 102 ms • 1.071 KB •

1

POST

2. Estudiante — inscripción, intento, submit y progreso / **Submit final — calificación servidor**

http://localhost:8080/api/v1/attempts/ed424030-91cd-4762-8651-a1c3764b0654/submit

PASS submit exitoso

PASS aprobado y porcentaje calculado por servidor

200

• 5728 ms • 791 B •

2

POST

2. Estudiante — inscripción, intento, submit y progreso / **Repetir MISMO submit — debe ser idempotente**

http://localhost:8080/api/v1/attempts/ed424030-91cd-4762-8651-a1c3764b0654/submit

PASS reintento idempotente devuelve 200

PASS sigue siendo el mismo intento

200

• 52 ms • 790 B •

2

GET

2. Estudiante — inscripción, intento, submit y progreso / **Consultar progreso del curso — debe quedar approved**

3. Casos adversos y control de acceso

Iteration 1

1

POST 3. Casos adversos y control de acceso / **Estudiante Intenta crear quiz — 403**
http://localhost:8080/api/v1/resources/8ed01545-021a-4131-ad89-4a98a8a760da/quiz

403 • 9 ms • 196 B • 1

PASS estudiante no puede autorar quiz

POST 3. Casos adversos y control de acceso / **Submit sin Idempotency-Key — debe fallar**
http://localhost:8080/api/v1/attempts/ed424030-91cd-4762-8651-a1c3764b0654/submit

400 • 6 ms • 198 B • 1

PASS sin Idempotency-Key falla

POST 3. Casos adversos y control de acceso / **Estudiante no puede crear badge**
http://localhost:8080/api/v1/courses/885fdeb-76eb-43dd-9e74-23117f658640/badge

403 • 7 ms • 196 B • 1

PASS 403 para estudiante

GET 3. Casos adversos y control de acceso / **Progreso sin token — debe fallar**
http://localhost:8080/api/v1/courses/885fdeb-76eb-43dd-9e74-23117f658640/progress

401 • 4 ms • 179 B • 1

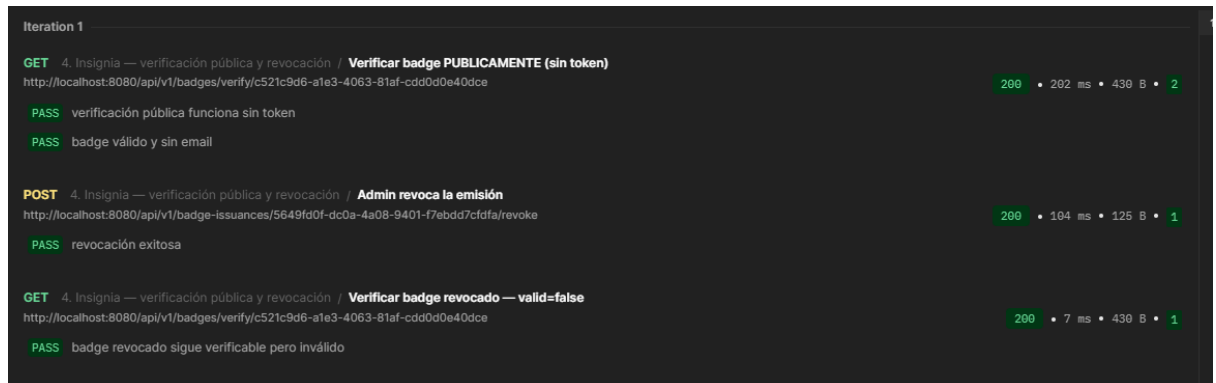
PASS sin token no accede a progreso

POST 3. Casos adversos y control de acceso / **Heartbeat fraudulento opcional (requiere progress_resource_id multimedia)**
http://localhost:8080/api/v1/resources/progress/heartbeat

400 • 8 ms • 188 B • 1

PASS posición multimedia imposible se rechaza

4. Insignia – verificación pública y revocación



```
Iteration 1

GET 4. Insignia — verificación pública y revocación / Verificar badge PUBLICAMENTE (sin token)
http://localhost:8080/api/v1/badges/verify/c521c9d6-a1e3-4063-81af-cdd0d0e40dce 200 • 202 ms • 430 B • 2

PASS verificación pública funciona sin token
PASS badge válido y sin email

POST 4. Insignia — verificación pública y revocación / Admin revoca la emisión
http://localhost:8080/api/v1/badge-issuances/5649fd0f-dc0a-4a08-9401-f7ebdd7cfdfa/revoke 200 • 104 ms • 125 B • 1

PASS revocación exitosa

GET 4. Insignia — verificación pública y revocación / Verificar badge revocado — valid=false
http://localhost:8080/api/v1/badges/verify/c521c9d6-a1e3-4063-81af-cdd0d0e40dce 200 • 7 ms • 430 B • 1

PASS badge revocado sigue verificable pero inválido
```

8. Limitaciones conocidas y trabajo pendiente

- La API y los workers se encuentran dockerizados; se verificó el escalamiento horizontal de workers ejecutando tres instancias simultáneas mediante Docker Compose.
- La API expone actualmente un único puerto del host, por lo que el escalamiento horizontal de varias réplicas de API requeriría incorporar un reverse proxy o balanceador.
- GetMediaStatus presenta una validación de autorización menos estricta que GetPlaybackURL, ya que no verifica todavía inscripción activa sobre el curso.
- No se implementó rate limiting ni protección CSRF específica.
- La API dispone de logs estructurados, pero no se implementaron métricas Prometheus, OpenTelemetry ni tracing distribuido.
- La cobertura de pruebas automáticas es desigual: identidad cuenta con go test, cursos/catálogo y evaluación/progreso/insignias cuentan con colecciones Postman, y multimedia presenta menor cobertura.
- Las pruebas de carga se ejecutaron hasta 2.000 usuarios virtuales. El sistema mantuvo buen comportamiento funcional en cargas menores, pero presentó degradación importante de latencia y errores a 2.000 VUs, por lo que ese nivel no puede considerarse actualmente una capacidad sostenida bajo los criterios definidos.
- El módulo de quizzes fue uno de los principales puntos de degradación identificados durante las pruebas de carga.
- La observabilidad avanzada, balanceo de múltiples instancias de API y optimización de consultas bajo alta concurrencia quedan como trabajo futuro.

9. Resultados de pruebas de carga con k6

Se ejecutaron cinco niveles independientes de carga: 100, 250, 500, 1000 y 2000 usuarios virtuales (VUs). Cada corrida distribuyó la carga entre escenarios de lectura pública, autenticación, administración, autoría, multimedia y aprendizaje del estudiante. Para evitar conflictos artificiales, los VUs del flujo de estudiante no compartieron la misma cuenta. Las métricas principales fueron: checks correctos, tasa de errores HTTP, número de requests y latencia p95 global y por módulo. Los thresholds del script se usaron como referencia, pero los resultados reportados corresponden a los valores observados.

9.1 Resultados consolidados

VUs	Requests	Checks	Error HTTP	p95 global	Interpretación
100	11,990	100.00%	0.000%	1.29 s	Estable funcionalmente
250	3,991	97.54%	2.36%	57.42 s	Degradación severa / corrida anómala
500	6,406	99.95%	0.047%	10.70 s	Correcto, pero con alta latencia
1000	8,156	99.93%	0.061%	21.48 s	Correcto, pero fuertemente degradado
2000	5,211	74.41%	26.60%	44.40 s	Colapso funcional y de latencia

9.2 Latencia p95 por módulo

VUs	Catálogo	Cursos	Auth	Admin	Media	Quizzes	Progreso
100	0.85 s	1.16 s	1.52 s	1.05 s	1.37 s	3.25 s	0.95 s
250	54.85 s	1.44 s	59.45 s	—	—	59.81 s	0.34 s
500	—	11.14 s	7.21 s	—	—	56.16 s	1.66 s
1000	10.56 s	24.33 s	16.48 s	12.95 s	30.90 s	59.99 s	—
2000	34.53 s	47.92 s	46.48 s	49.70 s	58.37 s	20.63 s	—

9.3 Observaciones de infraestructura con Docker Stats

Durante las corridas se tomaron muestras de docker stats sobre Redis, PostgreSQL, Mailpit, MinIO y ClamAV. Estas muestras no están etiquetadas por nivel de VU, por lo que se utilizan como evidencia cualitativa del comportamiento de los servicios y no como una serie temporal exacta.

Servicio	CPU observada	Memoria	% memoria	Lectura
Redis	≈ 0.5%–10.6%	≈ 11–13 MiB	≈ 0.38%–0.44%	Uso de memoria muy bajo; picos moderados de CPU.
PostgreSQL	≈ 0%–20.1%	≈ 39–72 MiB	≈ 1.34%–2.43%	Presentó los picos de CPU más claros durante actividad de datos.
MinIO	≈ 0%–15.5%	≈ 113–126 MiB	≈ 3.83%–4.26%	Picos de CPU asociados a operaciones multimedia; memoria estable.
Mailpit	≈ 0%–13.2%	≈ 26–33 MiB	≈ 0.90%–1.12%	Picos puntuales, sin presión sostenida de memoria.
ClamAV	≈ 0%–13.4%	≈ 784 MiB–1.04 GiB	≈ 26.6%–36.0%	Principal consumidor de

				memoria del entorno.
--	--	--	--	----------------------

No se observó agotamiento de memoria en PostgreSQL, Redis o MinIO. ClamAV mantuvo un consumo de memoria significativamente mayor, cercano a un tercio del límite de Docker, aun cuando su CPU fue generalmente baja. PostgreSQL registró picos de CPU cercanos al 20% y MinIO/Redis picos aproximados entre 10% y 15%. Por tanto, los altos p95 no pueden atribuirse únicamente a memoria agotada en estos contenedores. También deben considerarse la API Go ejecutada fuera de este listado, el pool de conexiones, bloqueos/consultas, límites de Docker Desktop y contención del host.

9.4 Análisis por nivel de carga

9.4.1. 100 VUs

La plataforma se mantuvo completamente estable funcionalmente: 11,990 solicitudes, 100% de checks correctos y 0% de errores HTTP. El p95 global fue de aproximadamente 1.29 s. Quizzes fue el módulo más costoso, con p95 de 3.25 s, mientras progreso se mantuvo por debajo de 1 s.

9.4.2. 250 VUs

Se observó una degradación atípicamente fuerte: p95 global de aproximadamente 57.42 s, 2.36% de errores HTTP y 97.54% de checks correctos. Auth, catálogo y quizzes se acercaron al timeout de 60 s. Dado que la corrida de 500 VUs presentó menos errores y menor p95 global, este resultado debe tratarse como una corrida anómala o afectada por condiciones transitorias del entorno.

9.4.3. 500 VUs

La integridad funcional fue muy alta: 99.95% de checks correctos y aproximadamente 0.047% de errores HTTP. Sin embargo, el p95 global aumentó a 10.70 s y quizzes llegó a 56.16 s, señalando un cuello de botella importante en ese flujo.

9.4.4. 1000 VUs

El sistema siguió respondiendo correctamente en la gran mayoría de las solicitudes, con 99.93% de checks correctos y 0.061% de errores HTTP. No obstante, el p95 global subió a 21.48 s. Quizzes se acercó a 60 s y también se degradaron courses, media, auth y admin.

9.4.5. 2000 VUs

A 2000 VUs se observó colapso claro de la configuración probada: los checks correctos bajaron a 74.41%, la tasa de error HTTP subió a 26.60% y el p95 global llegó a 44.40 s. Varios módulos presentaron p95 entre 34 s y 58 s. Por tanto, esta configuración local no cumple de forma estable el objetivo de 2000 usuarios concurrentes.

9.5 Conclusiones

La plataforma es funcionalmente estable con 100 VUs concurrentes en el entorno local probado. Entre 500 y 1000 VUs la tasa de error sigue siendo muy baja, pero la latencia deja de ser adecuada para una experiencia interactiva.

Quizzes es el cuello de botella más consistente y debe priorizarse para optimización.

A 2000 VUs la configuración local presenta degradación crítica tanto en errores como en latencia.

La corrida de 250 VUs no sigue una progresión monotonica y conviene repetirla para confirmar su comportamiento.

Los snapshots de Docker muestran que los contenedores de Redis, PostgreSQL y MinIO no agotaron memoria; por ello, la degradación probablemente involucra otros factores como la API, pool de conexiones, consultas, bloqueos y límites del host.

ClamAV fue el servicio con mayor consumo de memoria, alrededor de 0.8–1.0 GiB, por lo que reduce la memoria disponible del entorno aunque no aparezca como el principal consumidor de CPU.

9.6 Evidencias

Las salidas JSON de k6 de cada nivel de carga deben conservarse en `k6/results/` junto con capturas o registros de `docker stats`. Estos artefactos permiten auditar y reproducir los resultados presentados en este documento.

10. Anexos

- Repositorio: <https://github.com/danielasolarte/Proyecto1-Desarrollo>
- Especificación OpenAPI 3.1: `api/openapi.yaml` (54 endpoints).
- Documentación por módulo: `docs/modulo-auth-admin.md`, `docs/modulo-cursos-catalogo.md`.
- Colecciones Postman: `postman/MOOC_Persona1.postman_collection.json`, `postman/MOOC_Persona4_Postman.json`.
- Documentación del módulo de evaluación, progreso e insignias: descripción de endpoints y reglas principales disponible en `api/openapi.yaml` y en la colección `postman/MOOC_Persona4_Postman.json`.
- Evidencia de pruebas Persona 4: capturas de ejecución de Postman correspondientes a setup, autoría, flujo del estudiante, casos adversos y verificación/revocación de insignias.
- Evidencia de carga: scripts y resultados disponibles en la carpeta `k6/`, incluyendo ejecuciones de 100, 250, 500, 1000 y 2000 usuarios virtuales.
- Evidencia de despliegue: Docker Compose con API, PostgreSQL, Redis, MinIO, Mailpit, ClamAV y múltiples instancias del worker.